



The Forgotten Document-Oriented Database Management Systems: An Overview and Benchmark of Native XML DODBMSes in Comparison with JSON DODBMSes

Ciprian-Octavian Truică^{a,*,1,2}, Elena-Simona Apostol^{a,*,2}, Jérôme Darmont^b,
Torben Bach Pedersen^c

^a Computer Science and Engineering Department, Faculty of Automatic Control and Computers, University Politehnica of Bucharest, Bucharest, Romania

^b Université de Lyon, Lyon 2, ERIC UR 3083, Lyon, France

^c Center for Data Intensive Systems, Aalborg University, Aalborg, Denmark

ARTICLE INFO

Article history:

Received 1 June 2020

Received in revised form 3 October 2020

Accepted 15 January 2021

Available online 26 January 2021

Keywords:

XML Database Management Systems
JSON Database Management Systems
Document-Oriented Database Management Systems
Benchmark

ABSTRACT

In the current context of Big Data, a multitude of new NoSQL solutions for storing, managing, and extracting information and patterns from semi-structured data have been proposed and implemented. These solutions were developed to relieve the issue of rigid data structures present in relational databases, by introducing semi-structured and flexible schema design. As current data generated by different sources and devices, especially from IoT sensors and actuators, use either XML or JSON format, depending on the application, database technologies that store and query semi-structured data in XML format are needed. Thus, Native XML Databases, which were initially designed to manipulate XML data using standardized querying languages, i.e., XQuery and XPath, were rebranded as NoSQL Document-Oriented Databases Systems. Currently, the majority of these solutions have been replaced with the more modern JSON based Database Management Systems. However, we believe that XML-based solutions can still deliver performance in executing complex queries on heterogeneous collections. Unfortunately nowadays, research lacks a clear comparison of the scalability and performance for database technologies that store and query documents in XML versus the more modern JSON format. Moreover, to the best of our knowledge, there are no Big Data-compliant benchmarks for such database technologies. In this paper, we present a comparison for selected Document-Oriented Database Systems that either use the XML format to encode documents, i.e., BaseX, eXist-db, and Sedna, or the JSON format, i.e., MongoDB, CouchDB, and Couchbase. To underline the performance differences we also propose a benchmark that uses a heterogeneous complex schema on a large DBLP corpus.

© 2021 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

1. Introduction

With the emergence of Big Data and the Internet of Things (IoT) and the increasing amount of semi-structured information generated daily, new technologies have arisen for storing, managing, and extracting information and patterns from such data.

* Corresponding authors at: Computer Science and Engineering Department, Faculty of Automatic Control and Computers, University Politehnica of Bucharest, Romania.

E-mail addresses: ciprian.truica@upb.ro (C.O. Truică), elena.apostol@upb.ro (E.S. Apostol), jerome.darmont@univ-lyon2.fr (J. Darmont), tbp@cs.aau.dk (T.B. Pedersen).

¹ These authors contributed equally to this article.

² Part of this work was done at the Department of Computer Science, Aarhus University, Aarhus, Denmark.

The new technologies for storing data have been labeled with the name NoSQL and were initially developed to solve very specific problems. Currently, they provide different trade-offs and functionality (e.g., choosing high-availability over consistency) to be as generic as their counterparts Relational Database Management Systems (RDBMSes). Due to the semi-structured nature of data, NoSQL Database Management Systems (DBMSes) have been classified based on the data model used for storing information [1], i.e., key-value, document-oriented, wide column, and graph databases.

In this paper, we particularly study NoSQL Document-Oriented Databases Systems (DODBMSes) that encode data using the XML or JSON formats. We further focus on two subcategories of DODBMSes with respect to the data model used to encode documents: i) DODBMSes that encode data using the XML format are Native XML Database Management Systems (XDBMSes), and ii) DODBMSes that encode data using the JSON format are JSON Database Management Systems (JDBMSes).

Ses that encode data using the JSON format are JSON Database Management Systems (JDBMSes).

The NoSQL DBMSes became very popular with the increasing need for data storage, management, and analysis systems that scale with the volume. To address these needs, many NoSQL DBMSes compromise consistency to offer high-availability, partition tolerance, improved analytics, and high-throughput. These features are also a requirement for real-time web applications and Big Data processing and analysis and are available in JDBMSes as well.

XDBMSes have started to emerge after the eXtensible Markup Language (XML) has been standardized and became the common format for exchanging data between different applications running on the Web. Their primary use was to facilitate secure storage and fast querying of XML documents. Besides their primary use, XDBMSes prove useful for OLAP (Online Analytical Processing) style analysis and decision support systems that incorporate a time dimension and encode data in the XML format [2], and thus removing the need of using ETL (Extract Transform Load) processes to transform XML documents into a relational model. XML query languages and technologies, including XDBMSes, had been around before the NoSQL trend, and have been forgotten during the Big Data hype. In the field of relational databases, XML format is used as a Data Type, e.g., Oracle, DB2, PostgreSQL, etc. Currently, with the rise of the NoSQL movement, XDBMSes have become a subcategory of DODBMSes. But, with the emergence of processing platforms that uses Big Data or IoT technologies, where the data are transferred over computer networks into formats such as XML and JSON, the XDBMSes can be seen as a viable solution for storing and manipulating computer-generated semi-structured data.

We hypothesize that the more classical XDBMSes may still be useful in the Big Data era. Thus, in this study we want to address and use as guidelines the following research questions:

- Q1:** Are XDBMSes absolute and should be replaced by JDBMSes?
- Q2:** Are XDBMSes a viable candidate for Big Data Management?
- Q3:** Do JDBMSes outperform XDBMSes when using complex filtering and aggregation queries with different scale factors, on large and heterogeneous datasets?

To test our hypothesis and answer our research questions, we consider the following research objectives: i) discuss XDBMSes and compare their capabilities and features with several popular JDBMSes solutions; ii) propose a benchmark that evaluates the current needs and workloads available in Big Data and compare performance between the selected DODBMSes; iii) evaluate the performance of the selected DODBMSes using complex filtering and aggregation queries with different scale factors, on large and heterogeneous datasets.

For testing and analyzing with our proposed benchmark, we utilize several XDBMSes and JDBMSes solutions, that are free to use, and their license does not forbid benchmarking. Thus, we chose BaseX, eXist-db, and Sedna as representatives XDBMSes systems and MongoDB, CouchDB, and Couchbase as JDBMSes solutions.

As a result of our research and as a response to **Q1**, we claim that the more classical XML based DODBMSes may still be useful in the Big Data era. To demonstrate this and answer **Q2**, we propose a new benchmark for comprehensive DODBMSes analysis using a large dataset. And thereby we present a qualitative and quantitative performance comparison between XDBMSes and the more modern JDBMSes to answer **Q3**.

This paper is structured as follows. Section 2 presents an overview of different NoSQL DBMSes models, surveys, and benchmarks. Section 3 offers an in-depth overview and comparison of DODBMSes, focusing on the XDBMSes and JDBMSes subcategories. Section 4 introduces the proposed benchmark specification and

discusses the data and workload models, while Section 5 discusses the database physical implementation and presents the description of the benchmark's queries. Section 6 thoroughly details the experiments performed on the selected DODBMSes using our benchmark and discusses the results in detail. Finally, Section 7 concludes the paper, summarizes the results, and provides future research perspectives.

2. Related works

The NoSQL Database Management Systems (DBMSes) emerged as an alternative to Relational Database Management Systems (RDBMSes) in order to store and process huge amounts of heterogeneous data. However, NoSQL DBMSes did not appear as a replacement for RDBMSes, but as a solution to specific problems that require additional features (e.g., replication, high-availability, etc.) that are not handled well by traditional means [3]. The reasons commonly given to develop and use NoSQL DBMSes are summarized as follows [4]: avoidance of unneeded complexity, high throughput, horizontal scalability, running on commodity hardware, avoidance of expensive object-relational mapping, lowering the complexity and the cost of setting up a cluster, compromising reliability for better performance, and adapting to the requirements of cloud computing.

The classifications used for NoSQL DBMSes usually are done by either taking into account the persistence model or the data and query model. Using the persistence model, NoSQL DBMSes are classified as follows [4]:

- i) In-Memory Databases [5] are very fast because the most current used data are stored in memory, with optional subsequent disk flushes triggered at given periods or when the in-memory data are not used. Evidently, the size of the currently in-use data that can be stored is limited to the amount of memory. This problem can be resolved using vertical scaling to some degree as there is a limit to the amount of memory a system can hold. Moreover, the durability may become a problem if data are lost between subsequent disk flushes or if data persistence is disabled. A solution to this problem is data replication.
- ii) Memtables and SSTables Databases [6] buffer operations in memory using a Memtable after they have been written to an append-only commit log to ensure durability. After a certain amount of writes the Memtable gets flushed to disk as a whole into a SSTable. These DBMSes have performance characteristics comparable to those of In-Memory Database but solve the durability problem.
- iii) B-trees Databases [7] use the B-tree self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in logarithmic time [8].

NoSQL DBMSes are also classified by using the data and query model as follows [1,9]:

- i) Wide Column Databases are used to store, retrieve, and manage data using column families. Each record can have different numbers of cells and columns, making a row sparse without storing NULLs.
- ii) Graph Databases are used to store, retrieve, and manage information using a graph. Therefore, an object is modeled as a node and the edges between nodes become the relationships between the objects.
- iii) Key-Value Databases (KVDBMSes) are data storage systems designed for storing, retrieving, and managing associative arrays, i.e., dictionaries or hash tables.
- iv) Document-Oriented Databases (DODBMSes) have evolved from KVDBMSes and are used to store, retrieve, and manage semi-

structured data, i.e., documents, encoded using JSON, BSON, XML, or YAML formats.

There are multiple surveys on NoSQL DBMSes, in the following phrases we present the most relevant ones for our analysis. Article [10] provides a comparison regarding the performance and flexibility of KVDBMSes and DODBMSes over RDBMSes. The NoSQL DBMSes prove to be a better choice for high throughput applications that require data modeling flexibility and horizontal scaling. The authors of [1] offer a classification by data models of NoSQL DBMSes, and also they present the current and most popular solutions. In [11], the authors make a comparison and overview of NoSQL data models, query types, concurrency controls, partitioning, and replication. Article [12] presents a top-down overview of the NoSQL database field and propose a comparative classification model that relates functional and non-functional requirements to techniques and algorithms employed in these systems. The authors of [13] provide an overview of XML data manipulation techniques employed in conventional and temporal XDBMSes and study the support of such functionality in mainstream commercial DBMSes. Unfortunately, the paper presents only a general discussion about XDBMSes and other DBMSes with XML manipulation capabilities, and also no evaluation is provided. Thus, we can conclude that none of these surveys present an in-depth discussion and comparison of different subcategories of DODBMSes.

In the literature there are many data-centric benchmarks for the Big Data distributed systems and NoSQL DBMSes that focus either on structured data or on specific applications, such as MapReduce-based applications, rather than on unstructured or variety. In [14], the authors present a comprehensive survey and analysis of benchmarks for different types of Big Data systems including NoSQL systems. The authors of [15] present a new benchmark for textual data for distributed systems including MongoDB. None of the current literature presents benchmarks for modern native XDBMSes.

XDBMSes benchmarks are application-oriented and domain-specific, e.g., OpenEHR XML medical records [16], XMark which contains documents extracted from electronic commerce sites and content providers [17] or Transaction Processing over XML (TPoX) [18] which simulates a financial multi-user workload with XML data conforming to the FIXML standard. These benchmarks are used for testing the performance of DBMSes that are capable of storing, searching, modifying and retrieving XML data. Unfortunately, the majority of these benchmarks use rather small collections. And even for the benchmarks where the XML or JSON document size is up to the order of Gigabytes (GBs), the contained information is mostly homogeneous. Our proposed benchmark solution uses large heterogeneous collections with over 6 million records to test the scalability, filtering, and aggregation performance of complex queries for the current native XDBMSes.

Based on the lack of current literature regarding XDBMS, in this paper, we analyze the performance and functionality of DODBMSes solutions, while focusing on two distinct subclasses that use JSON or XML formats to encode data.

3. Document-Oriented Databases

Document-Oriented Databases Management Systems (DODBMSes) have evolved from Key-Value Databases [1]. DODBMSes are used for storing, retrieving, and managing semi-structured data. They have a schema-less flexible data representation, thus providing more flexibility for data modeling [19]. DODBMSes use documents for storing data such as XML or JSON. The flexibility provided by XML and JSON makes it easier to manipulate the information than it is for tables in Relational Database Management Systems (RDBMSes). Usually, documents are stored in collections. A

Native XML Database Management System (XDBMS) uses the XML (eXtensible Markup Language) data structure to encode documents and defines a hierarchical logical model based on the elements of this markup language [20,21]. A JSON Database Management System (JDBMS) uses the JSON structure for modeling documents and storing them in collections.

In DODBMSes, labels are used in storing the information. These labels describe the data and values in a record. New information can be added directly to a record without the need to modify the entire schema, as is the case for RDBMSes.

One of the benefits of using a DODBMS solution is the flexibility of modeling the data [22]. Data from the web, mobile, social, and IoT devices change the nature of the application's data model. In an RDBMS, these changes impose the modification of the schema by altering tables and adding or removing columns. Whereas, the flexibility of DODBMSes eliminates the need to force-fit the data into predefined attributes and tables.

Another benefit of a DODBMS is the fast write performance. Some DODBMSes prioritize high availability over strict data consistency. This ensures that both read and write operations will always be executed even if there is a hardware or network failure. In case of failure, the replication and eventual consistency mechanisms ensure that the environment will function.

Fast query performance is another benefit of a DODBMS. Most DODBMSes provide powerful query engines for CRUD (Create, Read, Update and Delete) operations and use indices and secondary indices to improve data retrieval. Additionally, the majority of DODBMS solutions support aggregation frameworks, either native or using MapReduce, for Data Analysis and Business Intelligence.

3.1. XDBMSes

In this subsection, we present several examples of XDBMSes that use standardized XPath and XQuery. Although there are multiple solutions of DBMSes that incorporate XML as data type (e.g., Oracle, PostgreSQL, DB2, MS SQL, etc. just to name a few), the majority of them fall out of the NoSQL movement. Furthermore, some have licenses that explicitly forbids benchmarking, e.g., commercial XDBMSes such as MarkLogic Server and Oracle Berkeley DB XML. Thus, for our comparison and benchmark, we chose the following three XDBMSes: BaseX, eXist-db, and Sedna.

BaseX

BaseX is an XDBMS written in Java that stores the data using a schema-free hierarchical model. Transactions in BaseX respect the ACID (Atomicity, Consistency, Isolation, and Durability) properties, enabling the concurrent access of multiple readers and writers [23]. Documents are stored either persistently on disk or in the main memory. BaseX uses a single instance environment, replication and data partitioning are not available.

BaseX provides CRUD operations and ad-hoc queries, including aggregation using XQuery 3.1 and XPath 3.1 [24]. Although, it works with various APIs such as XML DB or JAX-RX, it was not designed to work with a MapReduce framework.

BaseX supports multiple structural and value indices [23]. Structural indices are automatically created and include: i) name indices to reference the names of all elements and attributes, ii) path indices to store distinct paths of the documents in the database, and iii) document indices to reference all document nodes. Value indices are user-defined. They include: i) text indices for documents' text nodes to improve the performance of exact and range queries, ii) attribute indices to speed up comparisons on attribute values, iii) token indices to improve the multi-token attribute values, and iv) full-text indices to normalized tokens of text nodes and speed up queries which contain text expressions.

eXist-db

eXist-db [25] is a XDBMS implemented in Java that stores documents in the XML format. It stores data in-memory using Document Object Model (DOM) trees.

Although eXist-db does not have support for database-level transaction control, it has transactions internally, transparent to the user, and also has a persistent journal that is used to ensure the durability and consistency of the stored data. The database consistency is done automatically or using a sanity checker to detect the inconsistencies or damages in the core database files [26].

eXist-db supports data primary-secondary replication, thus allowing applications to be distributed over multiple servers through the use of Java Message Service (JMS) API. Although replication is available, data partitioning or sharding and distributing queries across multiple servers are not.

eXist-db provides CRUD operations and ad-hoc queries for filtering and aggregation using XQuery 3.1 and XPath 3.1 [24]. Unfortunately, it does not have the MapReduce functionality, which would offer more flexibility to the aggregation queries.

eXist-db supports four types of indices [27]: i) range indices that provide range and field-based searches, ii) text indices for full-text search, iii) n-gram indices for improving the performance of n-gram search, and iv) spatial indices for querying data using geometric characteristics, although this feature is currently experimental.

Sedna

Sedna is an XDBMS written in C that stores documents in the XML format [28]. Sedna provides ACID transactions, indexing, and persistent storage [29]. It uses the main memory to improve query performance [30]. Replication and partitioning are not implemented in Sedna.

Like the other XDBMSes, Sedna provides CRUD operations and ad-hoc queries for filtering and aggregation using XQuery 1.1 and XPath 2.0. However, it does not provide MapReduce functionality in working with these queries.

Value indices are used to index elements' content and attributes. Full-text indices can be created in Sedna to facilitate full-text search using XQuery.

3.2. JDBMSes

DODBMSes are designed for storing, retrieving, managing, and processing semi-structured data in the form of document. With the rise of the NoSQL movement, multiple DODBMS solutions, both proprietary and open-source, have been implemented. An important subcategory of these systems is JDBMS, which consists of systems that use the JSON format for document encoding. For our comparison, we choose three of the more popular and open source JDBMSes³: MongoDB, CouchDB, and Couchbase.

MongoDB

MongoDB is a DODBMS developed in C++ that focuses on combining the critical capabilities of RDBMSes with the innovations of NoSQL DBMSes. MongoDB uses a flexible, dynamic schema to store data. A record is stored in a document and multiple documents are stored in a collection. Documents in a collection do not necessarily have the same structure and so the number of attributes and their data type can differ from one record to another. In practice documents usually model objects from a high-level programming language. Although the database allows documents with a different number of attributes and different data types for the

same attributes, records have almost the same structure in a collection [31].

MongoDB stores the data in BSON documents. A BSON is a binary-encoded serialization of JSON-like documents. This format is easily parsed and lightweight with respect to the overhead needed to store data.

Transactions in MongoDB respect the BASE (Basically Available, Soft state, Eventual consistency) transaction model which ensures that all the modification operations will propagate on all the nodes in an asynchronous way. MongoDB uses Causal Consistency that enables operations to logically depend on preceding operations [32] and in-memory functionalities to improve the query execution time. Furthermore, this JDBMS supports multi-document transactions with ACID data integrity guarantees.

To achieve redundancy and data availability, MongoDB uses Replica Sets for primary-secondary replication. A replica set is a group of MongoDB instances that store the same dataset. To partition the data and distribute it across multiple machines, MongoDB uses Sharding. Sharding is a horizontal scaling mechanism that partitions and balances the data on multiple nodes or replica sets.

MongoDB supports CRUD operations and ad-hoc querying through the use of a JavaScript API available in the MongoDB client. The Aggregation Pipeline framework is a multi-stage pipeline that transforms documents into aggregated results using the concepts of data processing pipelines. Aggregation can also be achieved using the MapReduce framework.

MongoDB supports primary and secondary indexing. These indices can be a single field, compound (multikey), geospatial, hashed, and text. Text indices enable full-text search.

CouchDB

CouchDB is an open-source DODBMS developed in Erlang that provides a schema-free model for storing self-contained data using the JSON format [33].

Transactions in CouchDB respect document-level ACID properties with Multi-Versioning Concurrency Control (MVCC) [34]. CouchDB relies on Eventual Consistency together with incremental replication to maintain the data consistency. CouchDB does not provide in-memory capabilities. CouchDB provides primary-primary and primary-secondary asynchronous replication. Sharding is used to distribute horizontally in a cluster the copies of each replica [35]. To resolve inconsistencies, CouchDB uses a conflict-flagging mechanism.

CouchDB supports CRUD operations and ad-hoc querying using a JavaScript API called Mango. For aggregation, CouchDB provides Views and MapReduce functionalities [36]. Indexing in CouchDB is achieved through the use of views. CouchDB provides two types of indices: JSON and text for full-text search support.

Couchbase

Couchbase is a highly-scalable DODBMS that stores documents using the JSON encoding. It offers high availability, horizontal scaling, and high transaction throughput [37].

Transactions in Couchbase respect the ACID properties and rely on Eventual Consistency and Immediate Consistency. Couchbase has in-memory capabilities and keeps records into buckets. The buckets are of the following type i) Couchbase buckets used to store data persistently and in-memory, ii) Ephemeral buckets used when persistence is not required, and iii) Memcached buckets used to cache frequently-used data and minimize the number of queries a database-server must perform.

Couchbase uses a shared-nothing architecture and provides primary-primary and primary-secondary as well as partitioning through the use of sharding. Couchbase scales horizontally in a cluster.

³ DB-Engines ranking <https://db-engines.com/en/ranking/document+store>.

Table 1
DODBMS comparison.

	BaseX	eXist-db	Sedna	MongoDB	CouchDB	Couchbase
DBMS type	XDBMS	XDBMS	XDBMS	JDBMS	JDBMS	JDBMS
Data format	XML	XML	XML	BSON (Binary JSON)	JSON	JSON
Implementation	Java	Java	C	C++	Erlang	C/C++, Go, Erlang
Transaction	ACID	Isolation safe	ACID	BASE Multi-document isolation	Document-level ACID with MVCC	ACID
Consistency	Transaction Consistency	Automatic consistency Sanity checker	Transaction Consistency	Causal Consistency	Eventual Consistency	Eventual Consistency Immediate Consistency
In-memory	Yes	Yes	Yes	Yes	No	Yes
Replication	No	Primary-Secondary	No	Primary-Secondary	Primary-Primary Primary-Secondary	Primary-Primary Primary-Secondary
Partitioning	No	No	No	Sharding	Sharding	Sharding
Ad-hoc queries	XQuery 3.1 XPath 3.1	XQuery 3.1 XPath 3.1	XQuery 1.0 XPath 2.0	JavaScript	Mango	N1QL JavaScript
MapReduce	No	No	No	Yes	Yes	Yes
Secondary indices	Yes	Yes	Yes	Yes	Yes	Yes
Geospatial indices	No	No	Yes	Yes	Yes	Yes
Text indices	Yes	Yes	Yes	Yes	Yes	Yes

Ad-hoc data querying is achieved using a JavaScript API or a SQL-like language, i.e., N1QL (Non-1NF Query Language) [38]. These languages enable Couchbase to have OLTP (Online Transaction Processing) CRUD operations and ETL (Extract Transform Load) capabilities [39].

JavaScript MapReduce Views can be developed and stored on the server-side to specify complex indexing and aggregation queries [40].

Couchbase provides multiple types of indices: [40] i) composite indices to index multiple attributes, ii) covering indices to index the information needed for querying without accessing the data, iii) filtered (partial) indices to index a subset of the data used by the WHERE clause, iv) function-based indices that compute the value of an expression over a range of documents, v) sub-document indices to index embedded structures, vi) incremental MapReduce views to index the results of complex queries that perform sorting and aggregation to support real-time analytics over very large datasets, vii) spatial views using Spatial MapReduce to index multi-dimensional numeric data, and viii) full-text indices used for full-text search capabilities.

3.3. DODBMSes Comparison

Table 1 summarizes the main features of the presented databases. BaseX, Sedna and Couchbase offer ACID compliant transactions in comparison with MongoDB that offers BASE compliant multi-document isolation transactions and CouchDB that offers document-level ACID with MVCC transactions. XDBMSes support transaction consistency while MongoDB and CouchDB support causal consistency and eventual consistency, respectively. Couchbase supports both eventual and immediate consistency. A disadvantage of XDBMSes is that they do not have replication or partitioning mechanisms, except for eXist-db which offers primary-secondary replication. An advantage of XDBMSes is the use of XQuery and XPath for querying the data which makes ad-hoc querying an easy task. Although XDBMSes support aggregation queries, they do not provide MapReduce frameworks as a result of the lack of distribution capabilities. Another advantage of XDBMSes is that they offer different types of indices, including text indices for full-text search. As can be seen from Table 1, the chosen JDBMS solutions also offer different types of indices, but in addition to JDBMS, the one used

in XDBMS systems can also be added on properties and paths, not only on keys and values.

4. Benchmark specifications

4.1. Data model

For our benchmark, we proposed a heterogeneous entity-relationship schema that can be easily expanded with more complex relationships and new entities. Fig. 1 presents the proposed schema. The model's entities are described below.

- *Authors* is the entity that stores information about authors. Besides the unique identifier for each author *AuthorID*, the attribute *Name* is used for storing the name of each author.
- *Records* contains information about the published work of one or more authors. It stores the *Title*, the *URL* for quick access on the web, and the publishing *Year*. The many-to-many relationship *WrittenBy* correlates each record with the authors. A record can be either published as a book (or book chapter) or as an article (conference or journal). The relationship *IsA* is used for denoting the sub-type of a record.
- *Books* is the first sub-type of a record. This entity stores the following information: i) the unique book identifier *ISBN*, ii) the pages of a record using the attribute *Pages*, iii) the book editors using the multi-variate attribute *Editors*, and iv) the type of a record of this sub-type, i.e., book or book chapter, using the attribute *Type*. The one-to-many relationship *PublishedBy* is used to correlate each record of sub-type *Book* to a *Publisher*.
- *Articles* is the second sub-type of a record. Besides the unique identifier of a record in this sub-type, the entity *Articles* stored information about i) the pages of a record using the attribute *Pages*, and ii) the type of a record of this sub-type, i.e., conference or journal article, using the attribute *Type*. The one-to-many relationship *PublishedIn* is used to correlate each article to a journal.
- *Journals* entity stores information about an article publication venue. The attributes are: i) *ISSN* used as the unique identifier, ii) *Type* used to determine if the publication is a journal, proceedings, or special issue, iii) *Title* used for keeping the title of the journal or the conference name, iv) *Volume* used to

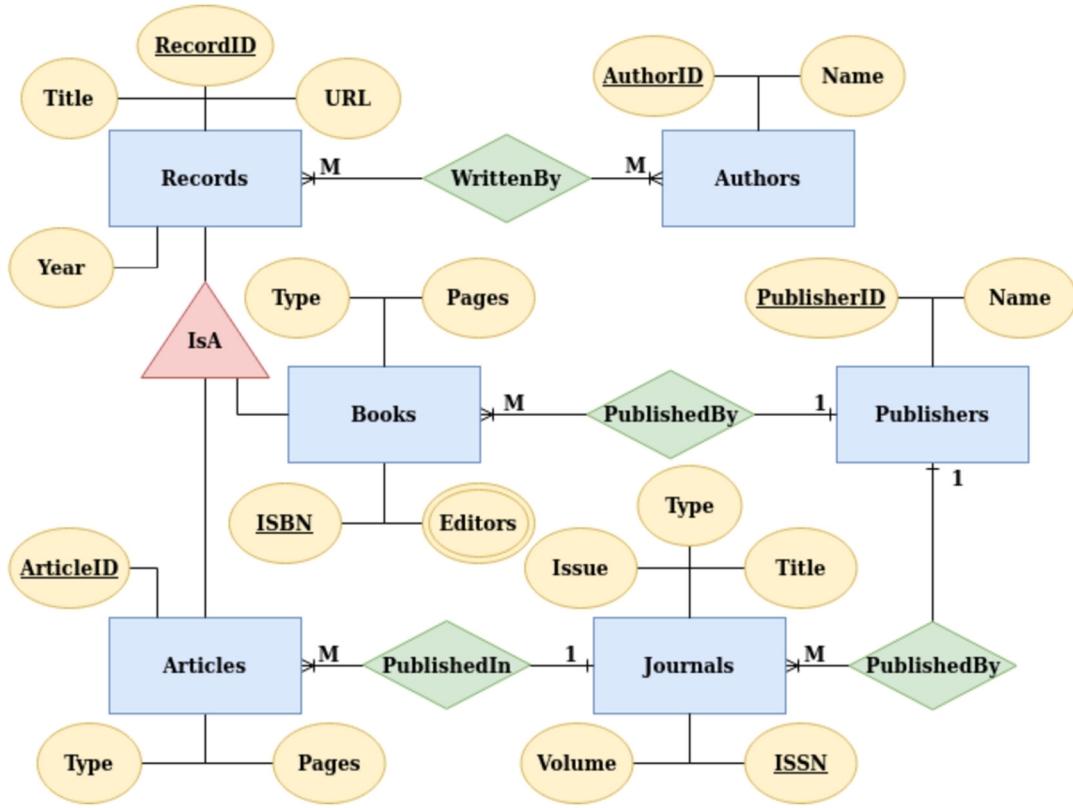


Fig. 1. Database entity-relationship diagram.

store the number of years since the first publication, and *v*) *Issue* used to store how many times the journal has been published during a year. The one-to-many relationship *PublishedBy* is used to correlate each record of sub-type *Journal* to a *Publisher*.

- *Publishers* is the entity that stores a unique identifier and the *Name* of a publishing house.

4.2. Workload model

The workload model follows two analysis directions: i) selection queries for filtering the corpus and extract subsamples, and ii) aggregation queries for creating reports.

For the selection queries, a constraint $c_1^i = \text{contains}(\text{Records.Title}, t_i)$ is used to extract the most relevant records that are contained in the title of a given set of terms. The constraint c_1^i utilizes the $\text{contains}(\cdot, \cdot)$ function, which verifies if a substring $t_i \in \{t | t \in \text{vocabulary}\}$ belongs to a string. In this case, the *vocabulary* is the set of terms extracted from each title using Tokenization.

Aggregation queries are used to create reports about the publishing activity of each author. These reports are created by counting the number of published records using attributes for grouping. To achieve this, we apply the aggregation operator γ_L with $L = (F, G)$, where F is the list of aggregation functions, and G is the list of attributes in the GROUP BY clause. We use the *Authors.Name* attribute in the GROUP BY clause to create an overview report of the publication activity for each author over his/her entire academic life. To determine the publishing patterns by year of each author, we use the *Records.Year* attribute that adds a time dimension to the previous report. For a more in-depth analysis of each published topic by author, we also use the c_1^i constraint to filter the dataset by keywords before counting the number of articles.

5. Benchmark implementation

5.1. Database design

The conceptual entity-relationship diagram described in Section 4 must be translated into the XML and JSON formats (Fig. 2). For the XML representation (Fig. 2a), the attributes of entities are directly encoded in the elements' names, e.g., the *Article.Type* is directly encoded into the *journal* label. In the case of the *Authors* entity, the records associated with the article are presented as multiple tags with the same name, i.e., *author*. For the JSON representation, the *Authors* entity becomes a list of values, i.e., the label *authors*. The information regarding an article is stored directly in the document using labels, e.g., type, publication year, etc. Using this representation, both schemes are greatly simplified and the need of relationships between entities disappears.

5.2. Query description

The proposed benchmark features nine queries with different complexity and selectivity, i.e., Q_1 to Q_9 . The first five queries are used to filter the dataset based on different constraints. Whereas, the last four queries are used to filter and group the data in order to obtain aggregated results.

5.2.1. Selection queries

The first set of queries selects the records that respect a given constraint.

The first query (Q_1^i) uses the constraint c_1^i to extract the documents which contain in their title a certain given term t_i (Equation (1)). The projection for the query, which specifies the set of selected attributes following the query execution, is $\Pi_1 = \{\text{Records.Title}\}$.

$$Q_1^i = \pi_{\Pi_1}(\sigma_{c_1^i}(\text{Records})) \quad (1)$$

<pre> <article mdate="2018-02-15" key="journals/vldb/ZhaoXLZW18"> { <author>Xiang Zhao</author> <author>Chuan Xiao</author> <author>Xuemin Lin</author> <author>Wenjie Zhang</author> <author>Yang Wang</author> <title>Efficient structure similarity searches: a partition-based approach.</title> <journal>VLDB J.</journal> <volume>27</volume> <number>1</number> <pages>53-78</pages> <year>2018</year> <ee>https://doi.org/10.1007/s00778-017-0487-0</ee> <url>db/journals/vldb/vldb27.html#ZhaoXLZW18</url> </article> </pre>	<pre> { "key": "journals::vldb::ZhaoXLZW18", "mdate": "2018-02-15", "authors": ["Xiang Zhao", "Chuan Xiao", "Xuemin Lin", "Wenjie Zhang", "Yang Wang"], "title": "Efficient structure similarity searches: a partition-based approach.", "type": "article", "journal": "VLDB J.", "volume": "27", "number": "1", "pages": "53-78", "year": "2018", "ee": "https://doi.org/10.1007/s00778-017-0487-0", "url": "db/journals/vldb/vldb27.html#ZhaoXLZW18" } </pre>
---	---

(a) XML Document

(b) JSON Document

Fig. 2. Document representation in XML and JSON.

The second query (Q_2^{ij}) extracts the records that contain in their title two terms (Equation (2)). It uses the constraint c_s^i , $s \in \{i, j\}$ with $i \neq j$. The query is written using the INTERSECTION operator between the results returned by Q_1^i for term t_i and Q_1^j for term t_j . Due to the nature of the filtering condition, we can concatenate the separate conditions to create a single conditional expression using the *and* logical operator ($\wedge$), i.e., $c_1^i \wedge c_1^j$. As in the case of the first query, the projection remains Π_1 .

$$\begin{aligned}
 Q_2^{ij} &= Q_1^i \cap Q_1^j \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i}(\text{Records})) \cap \pi_{\Pi_1}(\sigma_{c_1^j}(\text{Records})) \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i \wedge c_1^j}(\text{Records}))
 \end{aligned} \quad (2)$$

Q_3^{ij} extracts the records that contain in their title at least one of the terms given through the c_1^i or c_1^j constraints, with $i \neq j$ (Equation (3)). The query is written using the UNION operator between the results returned by Q_1^i for term t_i and Q_1^j for term t_j . The projection remains Π_1 . As for query Q_2^{ij} , the conditions can be concatenated to create a single conditional expression using the *or* logical operator ($\vee$), i.e., $c_1^i \vee c_1^j$.

$$\begin{aligned}
 Q_3^{ij} &= Q_1^i \cup Q_1^j \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i}(\text{Records})) \cup \pi_{\Pi_1}(\sigma_{c_1^j}(\text{Records})) \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i \vee c_1^j}(\text{Records}))
 \end{aligned} \quad (3)$$

The fourth query (Q_4) filters the *Records* entity and extracts the documents that contain in their title the terms t_i , t_j , and t_k (Equation (4)). As for the previous queries, the projection attributes are given using Π_1 . The query is written using the INTERSECTION operator between the results obtained by Q_1^i , Q_1^j , and Q_1^k for terms t_i , t_j , and t_k respectively. Due to the nature of the filtering conditions, they can be concatenated into one constraint $c_1^i \wedge c_1^j \wedge c_1^k$.

$$\begin{aligned}
 Q_4^{ijk} &= Q_1^i \cap Q_1^j \cap Q_1^k \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i}(\text{Records})) \cap \pi_{\Pi_1}(\sigma_{c_1^j}(\text{Records})) \\
 &\quad \cap \pi_{\Pi_1}(\sigma_{c_1^k}(\text{Records})) \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i}(\text{Records}) \cap \sigma_{c_1^j}(\text{Records}) \cap \sigma_{c_1^k}(\text{Records})) \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i \wedge c_1^j \wedge c_1^k}(\text{Records}))
 \end{aligned} \quad (4)$$

The last selection query (Q_5) extracts the documents that contain in their title one or more of the searched terms t_s , $s \in \{i, j, k\}$

with $i \neq j \wedge i \neq k \wedge j \neq k$. The query is written using the UNION operator between the results obtained by each Q_1^s for t_s terms. The nature of the filtering constraints permit the query to be written using one constraint $c_1^i \vee c_1^j \vee c_1^k$ and the projection Π_1 (Equation (5)).

$$\begin{aligned}
 Q_5^{ijk} &= Q_1^i \cup Q_1^j \cup Q_1^k \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i}(\text{Records})) \cup \pi_{\Pi_1}(\sigma_{c_1^j}(\text{Records})) \\
 &\quad \cup \pi_{\Pi_1}(\sigma_{c_1^k}(\text{Records})) \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i}(\text{Records}) \cup \sigma_{c_1^j}(\text{Records}) \cup \sigma_{c_1^k}(\text{Records})) \\
 &= \pi_{\Pi_1}(\sigma_{c_1^i \vee c_1^j \vee c_1^k}(\text{Records}))
 \end{aligned} \quad (5)$$

5.2.2. Aggregation queries

The last four queries use aggregation to count the number of articles using different filtering constraints and attributes in the GROUP BY clause.

The sixth query (Q_6) uses aggregation to determine the number of articles written by each author (Equation (6)). It uses a JOIN operation between the *Records* and *Authors* entities. Because there is a many-to-many relationship between the two entities, the JOIN also traverses *WrittenBy*. The projection attributes are $\Pi_6 = \{\text{Author.Name}, \text{count}\}$. To determine the number of articles for each author, we use the aggregation operator γ_{L_6} , where $L_6 = (F_6, G_6)$. The list of aggregation functions is given by F_6 , while the set of attributes in the GROUP BY clause is given by G_6 . The list of aggregation functions is $F_6 = \{\text{count}(\text{Records.RecordID})\}$, where the *count* is the counting aggregation function. The set of attributes in the GROUP BY clause is $G_6 = \{\text{Authors.Name}\}$.

$$Q_6 = \pi_{\Pi_6}(\gamma_{L_6}(\text{Authors} \bowtie \text{Records})) \quad (6)$$

The seventh query (Q_7) counts the number of articles published by an author for each year (Equation (7)). The query makes use of a JOIN operation between the *Records* and *Authors* entities, as in the case of query Q_6 . The projection uses the following attributes $\Pi_7 = \{\text{Author.Name}, \text{Record.Year}, \text{count}\}$. To determine the number of articles written in a year by each author, we use the aggregation operator γ_{L_7} , where $L_7 = (F_7, G_7)$. For query Q_7 , the list of aggregation functions is given by F_7 , while the set of attributes in the GROUP BY clause is given by G_7 . The list of aggregation functions is $F_7 = \{\text{count}(\text{Records.RecordID})\}$, where the *count* is the counting function used for determining the number of articles written in a year by each author. The set of attributes in the GROUP BY clause is $G_7 = \{\text{Authors.Name}, \text{Records.Year}\}$.

$$Q_7 = \pi_{\Pi_7}(\gamma_{L_7}(\text{Authors} \bowtie \text{Records})) \quad (7)$$

The eighth query (Q_8) extracts the documents that contain in their title all of the searched terms, and then it counts the number of articles grouped by author and year. As in the case of Q_6 , the JOIN operation is between the *Records* and *Authors* entities. The query is written using the INTERSECTION operator. The filtering is done using the constraints c_1^i, c_1^j, c_1^k which ensures that the title contains all terms t_i, t_j , and t_k with $i \neq j \wedge i \neq k \wedge j \neq k$. The projection attributes and the aggregation operator remains the same as in the case of Q_7 , i.e., Π_7 and γ_{L_7} . Due to the nature of the filtering conditions, the query can be rewritten using only one constraint $c_1^i \wedge c_1^j \wedge c_1^k$.

$$\begin{aligned} Q_8 &= \pi_{\Pi_7}(\gamma_{L_7}(\sigma_{c_1^i}(\text{Records} \bowtie \text{Authors}) \\ &\quad \cap \sigma_{c_1^j}(\text{Records} \bowtie \text{Authors}) \cap \sigma_{c_1^k}(\text{Records} \bowtie \text{Authors}))) \\ &= \pi_{\Pi_7}(\gamma_{L_7}(\sigma_{c_1^i \wedge c_1^j \wedge c_1^k}(\text{Records} \bowtie \text{Authors}))) \end{aligned} \quad (8)$$

The last query (Q_9) extracts the documents that contain in their title one or more of the searched terms $t_s, s \in \{i, j, k\}$ and $i \neq j \wedge i \neq k \wedge j \neq k$, by filtering through the use of constraint c_1^s . The JOIN operator is used once again between the *Records* and *Authors* entities, as in the case of Q_6 . The projection attributes and the aggregation operator remain the same as in the case of Q_7 , i.e., Π_7 and γ_{L_7} . The filtering constraints c_1^i, c_1^j, c_1^k are applied on the *Records* entity. The query uses the UNION operator between the relationship obtained after filtering. Due to the nature of the filtering, the query can be rewritten using one constraint $c_1^i \vee c_1^j \vee c_1^k$.

$$\begin{aligned} Q_9 &= \pi_{\Pi_7}(\gamma_{L_7}(\sigma_{c_1^i}(\text{Records} \bowtie \text{Authors}) \\ &\quad \cup \sigma_{c_1^j}(\text{Records} \bowtie \text{Authors}) \cup \sigma_{c_1^k}(\text{Records} \bowtie \text{Authors}))) \\ &= \pi_{\Pi_7}(\gamma_{L_7}(\sigma_{c_1^i \vee c_1^j \vee c_1^k}(\text{Records} \bowtie \text{Authors}))) \end{aligned} \quad (9)$$

6. Experiments

6.1. Experimental conditions

All tests were run on an IBM System x3550 M4 with 64 GB of RAM, and an Intel(R) Xeon(R) CPU E5-2670 v2 @ 2.50 GHz. The XDBMSes used for benchmarking are BaseX, eXist-db, and Sedna. For comparison reasons we also use three JDBMSes: MongoDB, CouchDB, and Couchbase. We chose these DODBMSes because they are free to use and because their licenses do not forbid benchmarking.

The versions of the deployed DODBMSes are listed in Table 2. The proposed benchmark, the results, and the used dataset are publicly available on-line.⁴

As the chosen XDBMS solutions do not have partitioning, we could not distribute them. Therefore, we deployed and tested them on a single instance environment. Moreover, for comparison reasons, we also used a single instance environment for MongoDB, CouchDB, and Couchbase.

The query parameterization is presented in Table 3. Each term t_i ($i = \overline{1,3}$) is used for filtering the records through the constraint $c_1^{(i)}$. Thus for the first set of queries, i.e., Q_1^i, Q_2^{ij} , and Q_3^{ijk} , the i, j , and k indices ($i \neq j \wedge i \neq k \wedge j \neq k$) represent the $i' \in \overline{1,3}$ index of the $t_{i'}$ used for filtering.

Table 2

Benchmarked DODBMSes.

DODBMS	Version
BaseX	9.3.3
eXist-db	5.2.0
Sedna	3.5
MongoDB	4.2.7
CouchDB	3.1.0
Couchbase	6.5.1

Table 3

Query parameter values.

Parameter	Value
t_1	database
t_2	text
t_3	mining

6.2. Dataset

The experiments are performed on 6 150 738 records extracted from DBLP.⁵ The initial dataset is split into 4 different subsets to test the scalability of each DODBMS w.r.t. the number of records. These subsets contain 768 842, 1 537 685, 3 075 369, and 6 150 738 records, respectively. Each subset allows scaling experiments and are associated with a scale factor SF parameter, where $SF = \{0.125, 0.25, 0.5, 1\}$. Table 4 presents the size of the 4 subsets, both as raw data and the resulting DODBMS collection dimension.

For all the XDBMSes as well as for CouchDB and Couchbase, we can observe that database size is larger than the raw dataset. This increase is a direct result of the overhead required by the DODBMSes to manage and store the data. MongoDB uses compression mechanisms, which in turn manage to decrease the database size by minimizing the overhead.

6.3. Query implementation

Data are stored within each DODBMS using a denormalized schema; thus, one-to-many and many-to-many relationships are encapsulated inside the same document. To achieve denormalization, JDBMSes employ nested documents, lists, and lists of nested documents, while XDBMSes use the hierarchical structure of the XML format. To normalize the information and apply filtering and aggregation operations and functions, we use the native syntax, operators, query language clauses, and frameworks provided by each DODBMS. Table 5 presents the implementation language and operators.

For the XDBMSes, we implemented the queries using XQuery. The aggregation queries for BaseX and eXist-db use the XQuery 3.1 syntax for sorting and grouping, i.e., FOR... WHERE... GROUP BY... ORDER BY... For Sedna, we use the XQuery 1.1 syntax for sorting and grouping, i.e., FOR... WHERE... LET... ORDER BY... We used the native Command Line Interfaces to run these queries.

The aggregation queries in MongoDB are implemented using its Aggregation Pipeline framework. To deal with nested documents, the unwind operator is used to flatten an array field of nested documents. This operator is useful when trying to normalize the one-to-many and many-to-many which through denormalization are stored in the JSON format as nested documents or lists of nested documents. We used the native Command Line Interfaces to run these queries.

CouchDB uses Materialized Views for aggregation and to deal with nested and list of nested documents. These views are im-

⁴ GitHub Sources <https://github.com/cipriantruica/The-Forgotten-DODBMSes>.

⁵ DBLP <http://dblp.org/>.

Table 4
Dataset.

SF	No. Records	Raw XML	Raw JSON	BaseX DB size	eXist-db DB size	Sedna DB size	MongoDB DB size	CouchDB DB size	Couchbase DB size
0.125	768 842	0.38GB	0.34GB	0.53GB	0.44GB	1.78GB	0.17GB	0.43GB	0.44GB
0.25	1 537 685	0.75GB	0.67GB	1.05GB	0.86GB	3.36GB	0.33GB	0.85GB	0.92GB
0.5	3 075 369	1.51GB	1.36GB	2.09GB	1.74GB	6.71GB	0.67GB	1.69GB	1.78GB
1	6 150 738	2.25GB	2.06GB	3.14GB	2.59GB	10.17GB	1.02GB	2.81GB	3.02GB

Table 5
Filtering and aggregation queries.

Database	Filtering Query	Aggregation Queries
BaseX	XQuery 3.1	XQuery 3.1 syntax for sorting and grouping
eXist-db	XQuery 3.1	XQuery 3.1 syntax for sorting and grouping
sedna	XQuery 1.0	XQuery 1.1 syntax for sorting and grouping
MongoDB	JavaScript	JavaScript Aggregation Pipeline with unwind operator
CouchDB	JavaScript/Mango	JavaScript/Mango Materialized Views
Couchbase	N1QL	N1QL with UNNEST clause

Table 6
Filter queries selectivity.

SF	Q_1^1	Q_1^2	Q_1^3	Q_2^{12}	Q_2^{13}	Q_2^{23}	Q_3^{12}	Q_3^{13}	Q_3^{23}	Q_4	Q_5
0.125	0.992	0.987	0.993	0.999	0.999	0.999	0.980	0.986	0.980	0.999	0.974
0.25	0.991	0.986	0.992	0.999	0.999	0.999	0.978	0.984	0.979	0.999	0.971
0.5	0.990	0.982	0.991	0.999	0.999	0.999	0.973	0.982	0.975	0.999	0.966
1	0.993	0.987	0.994	0.999	0.999	0.999	0.981	0.988	0.982	0.999	0.976

plemented using CouchDB's MapReduce framework. The mapper function is used to flatten nested documents and filter the field. The reducer function is used for applying an aggregation function and returning the final result. We used cURL to run these queries.

To manipulate nested array in Couchbase, N1QL offers developers the UNNEST clause. This clause is used to flatten the arrays in the parent document. Thus, the UNNEST clause conceptually performs a JOIN operation between nested arrays and the parent document. As data are stored using the JSON format, the JOIN operation increases the runtime and decreases the overall retrieval performance. For Couchbase, we used the native Command Line Interfaces to run these queries.

6.4. Query selectivity

Selectivity, i.e., the amount of retrieved data ($n(Q)$) w.r.t. the total amount of available data (N), depends on the number of attributes in the WHERE and GROUP BY clauses. The selectivity formula used for a query Q is $S(Q) = 1 - \frac{n(Q)}{N}$. For the selection queries, we set N equal to the cardinality of the *Records* entity, i.e., $N = ||Records||$. Table 6 presents the filtering queries' selectivity w.r.t. the *SF*. The queries with more restrictive conditions return a smaller number of records and the selectivity is higher, e.g., Q_2^{ij} . The queries with more inclusive restrictions return a higher number of records and the selectivity is lower, e.g., Q_3^i .

For the aggregation queries, we set N equal to the number of queries returned by joining the entities *Records* with *Authors*, i.e., $N = ||Authors \bowtie Records||$. Table 7 shows the aggregation queries' selectivity w.r.t. the *SF* factor. Query Q_8 is the most restrictive query. Because of the filtering and grouping conditions, Q_8 returns a small number of records, and its selectivity is almost equal to 1. The most inclusive query is Q_7 , and it has a low selectivity w.r.t. *SF*. Because of the less restrictive filtering and grouping conditions, the selectivity of this query is less than 0.45. The selectivity of Q_6 increases w.r.t. *SF*, meaning that the number of records returned by the query increases more gradually than the size of the corpus.

Table 7
Aggregation queries selectivity.

SF	Q_6	Q_7	Q_8	Q_9
0.125	0.651	0.256	0.999	0.974
0.25	0.728	0.345	0.999	0.970
0.5	0.797	0.448	0.999	0.969
1	0.848	0.424	0.999	0.974

6.5. Performance metrics and execution protocol

We use the query response time as the only metric for the benchmark. It is symbolized for each query by $t(Q_i^*) \forall i \in [1, 9]$. All queries are executed 10 times, which is sufficient according to the central limit theorem. Additionally, all executions are warm runs, i.e., either caching mechanisms must be deactivated, or a cold run where each query must be executed once (but not taken into account in the benchmark's results) to fill in the cache. Queries must be written in the native scripting language of the target DODBMS and executed directly inside the specified system using the command line interpreter. Lastly, the average response time and standard deviation are computed for each $t(Q_i^*)$.

6.6. Results

Fig. 3 presents the results of Q_1^i where $i = \overline{1, 3}$ is used to denote the keyword t_i . MongoDB and BaseX offer the fastest time performance among the DODBMSes that encode documents using JSON and XML, respectively, regardless of the keyword w.r.t. *SF*. For Q_1^2 query which has the lowest selectivity of the three Q_1^i queries, the time performance of CouchDB is with a factor of $\sim 2x$ faster than eXist-db w.r.t. *SF*. The time performance of CouchDB and eXist-db for Q_1^1 and Q_1^3 tend to become the same w.r.t. *SF*, i.e., the performance difference factor between CouchDB and eXist-db at *SF* = 0.125 is $\sim 0.8x$ which increases to $\sim 0.9x$ for *SF* = 1. CouchDB time performance is with a factor of $\sim 1.1x$ faster than Couchbase for all the Q_1^i queries regardless of *SF*. Couchbase and eXist-db have similar performance for query Q_1^3 and *SF* = 1. Sedna

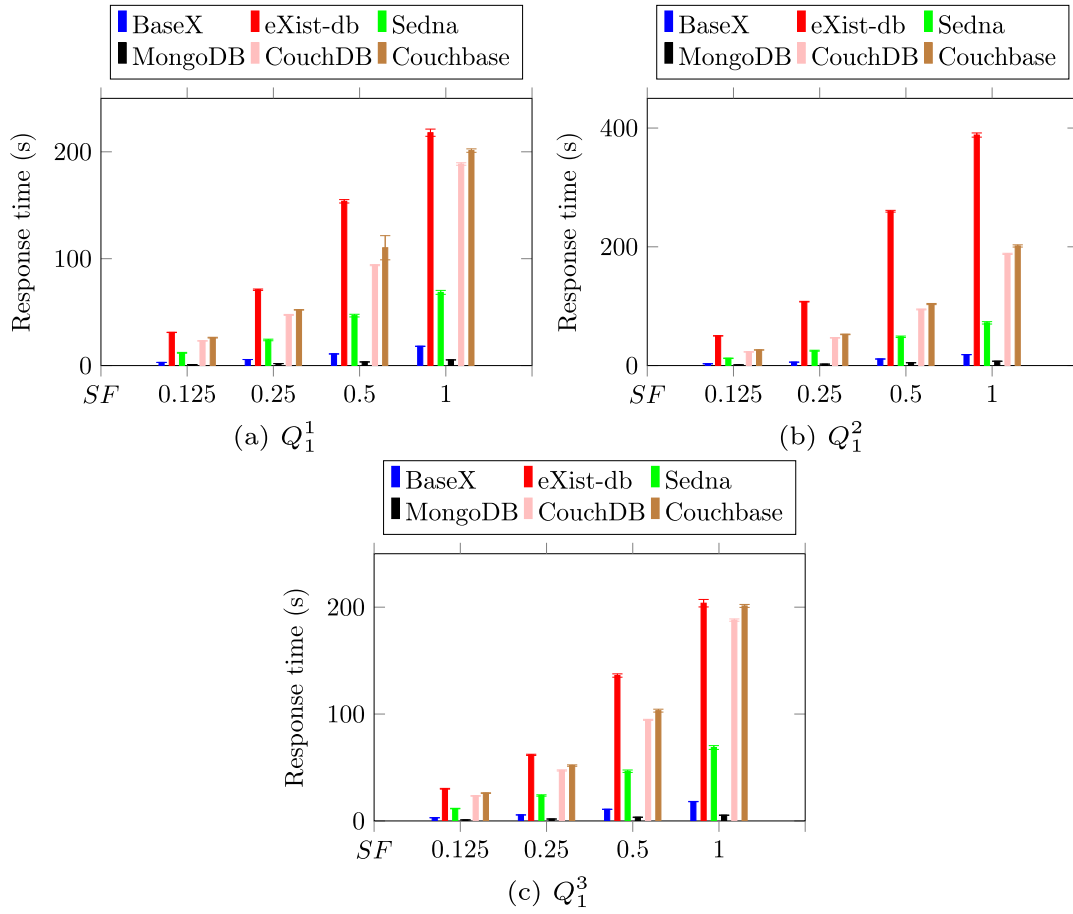


Fig. 3. Response time for Q_1^i . (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

performance is almost constant regardless of query selectivity w.r.t. SF . The overall best performance is achieved by MongoDB.

Fig. 4 presents the results of Q_2^{ij} and Q_3^{ij} queries where i and j indicate the t_i and t_j keywords used for filtering (Table 3) with $i = \overline{1, 3}$, $j = \overline{1, 3}$, and $i \neq j$. For this set of queries, MongoDB has the best overall time performance regardless of the SF factor. BaseX achieves the second overall best performance and the best performance among the tested XDBMSes, regardless of the SF . For the Q_2^{ij} set of queries, the time performance of MongoDB has a factor between $\sim 3.2x$ and $\sim 3.6x$ faster than BaseX w.r.t. SF . For the Q_3^{ij} set of queries, the time performance of MongoDB has a factor between $\sim 1.8x$ and $\sim 2.2x$ faster than BaseX w.r.t. SF .

Couchbase presents the highest execution time for the Q_2^{ij} queries regardless of SF , followed by the execution time of CouchDB. CouchDB time performance is with a factor of $\sim 1.2x$ and $\sim 1.1x$ faster than Couchbase for the Q_2^{ij} , respectively Q_3^{ij} queries regardless of SF . The eXist-db XDBMS has the worst performance for the Q_3^{ij} set of queries regardless of the SF . For the Q_2^{ij} set of queries, Sedna time performance has a factor of $\sim 2x$ better than CouchDB and a factor of $2x$ worse than eXist-db. For the Q_3^{ij} set of queries, Sedna's query execution time is with a factor of $\sim 1.5x$ better than CouchDB and with a factor of $\sim 5x$ worse than BaseX.

Fig. 5 presents the time performance of Q_4 and Q_5 queries for each DODBMS w.r.t. SF . The time performance trend for Q_4 and Q_5 remains similar to the ones for Q_2^{ij} and Q_3^{ij} , respectively. CouchDB time performance is with a factor of $\sim 1.3x$ and $\sim 1.2x$ faster than Couchbase for the Q_2^{ij} , respectively Q_3^{ij} queries regardless of SF . MongoDB achieves the overall best time performance

for both queries. BaseX has the second-best time performance among the tested DODBMSes and the best performance among the XDBMSes.

Fig. 6 shows the results for the aggregation queries, i.e., Q_6 to Q_9 . For the queries Q_6 , Q_7 , and Q_9 , BaseX has the best time performance and significantly outperforms MongoDB and CouchDB with a factor of $\sim 2x$, regardless of the SF . For the Q_8 query, CouchDB achieved the best query execution time, while Couchbase the worst. MongoDB has the second best query response time among the studied DODBMSes for Q_6 , Q_7 , and Q_9 . MongoDB's response time for these queries is almost on parity with the response time of CouchDB w.r.t. SF , although MongoDB executes the aggregation functions at runtime.

For Q_7 , Couchbase has a large standard deviation. During testing, this query finished with the error "Index scan timed out". The tests that finished with the status "success" returned fluctuating time performance for each run. This abnormal behavior of the Couchbase system can be sometimes observed for complex queries on large collections.

For Q_8 which has the highest selectivity, CouchDB holds the best time performance. We attribute this result to the mechanism used by CouchDB to store aggregation functions. Aggregation functions are stored in materialized views also named indices in CouchDB. Using this technique, CouchDB manages to outperform BaseX and MongoDB, which execute aggregation functions at runtime, for queries with high selectivity. With Couchbase, the complexity and selectivity together with the UNNEST clause required to extract the nested documents in order to filter and group the information, increases the runtime significantly while decreasing the overall query performance.

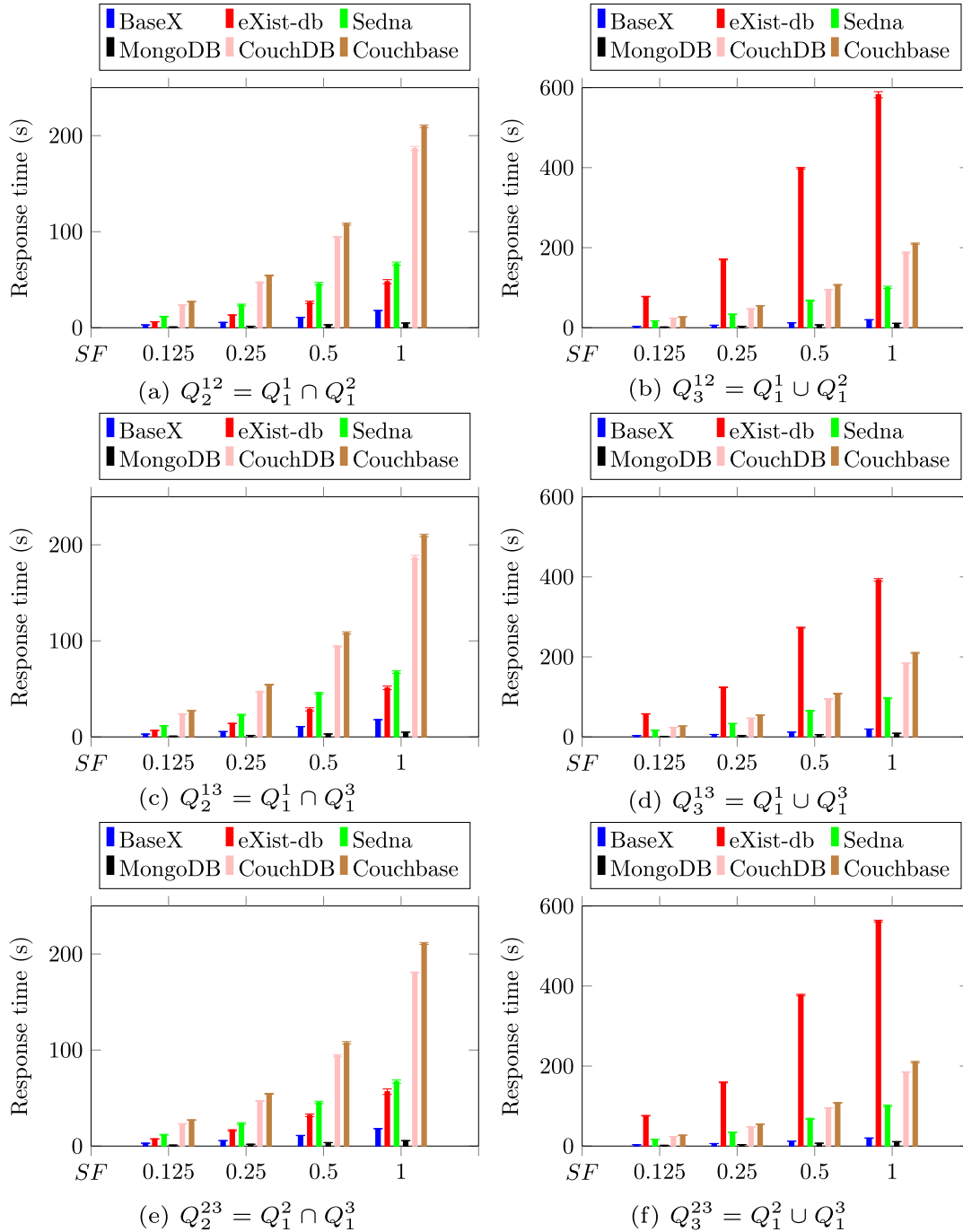


Fig. 4. Response time for Q_2^{ij} and Q_3^{ij} . (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

The aggregation queries did not work on Sedna. When executing these queries, the XDBMS remained unresponsive for days, and we had to manually stop the system, the related services, and the background processes. We note that Sedna also executes aggregation functions at runtime. We suspect that one reason for Sedna's failure to execute the aggregation queries is also the outdated XQuery 1.0 query language.

The eXist-db XDBMS has the highest query time for Q_6 , Q_7 , and Q_9 queries. The execution is done at runtime. For this XDBMS, query Q_7 worked only for $SF = 0.125$. For other SF values, the query returned memory errors, although we have tuned this XDBMS to work with the same parameters as the other DODBMSes. Thus, eXist-db is highly dependent on the JVM (Java Virtual Machine) memory allocation mechanism.

6.7. Discussions on the experimental design choices

In this study, we present our findings regarding the performance of filtering and aggregation queries on a large dataset for XDBMSes and JDBMSes w.r.t. different scale factors. We observe that the XDBMSes perform as well as JDBMSes for specific use cases, with BaseX even outperforming the more popular JDBMSes on three out of the four aggregation queries. Among the JDBMSes, MongoDB has the overall best performance.

For our comparison, we do not take into account horizontal scalability through sharding and replication, as not all of the analyzed DBMSes have such a functionality. Furthermore, it is essential first to understand single-node performance before considering

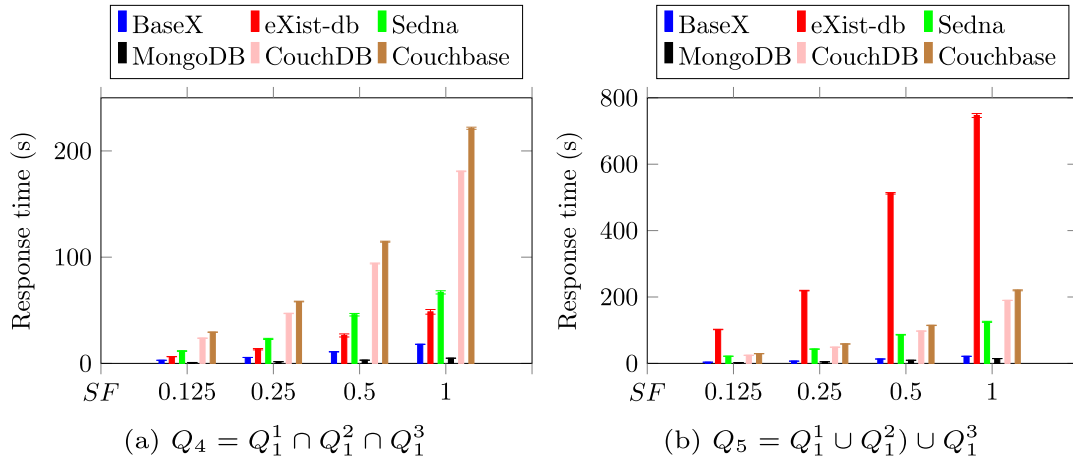


Fig. 5. Response time for Q_4 and Q_5 . (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

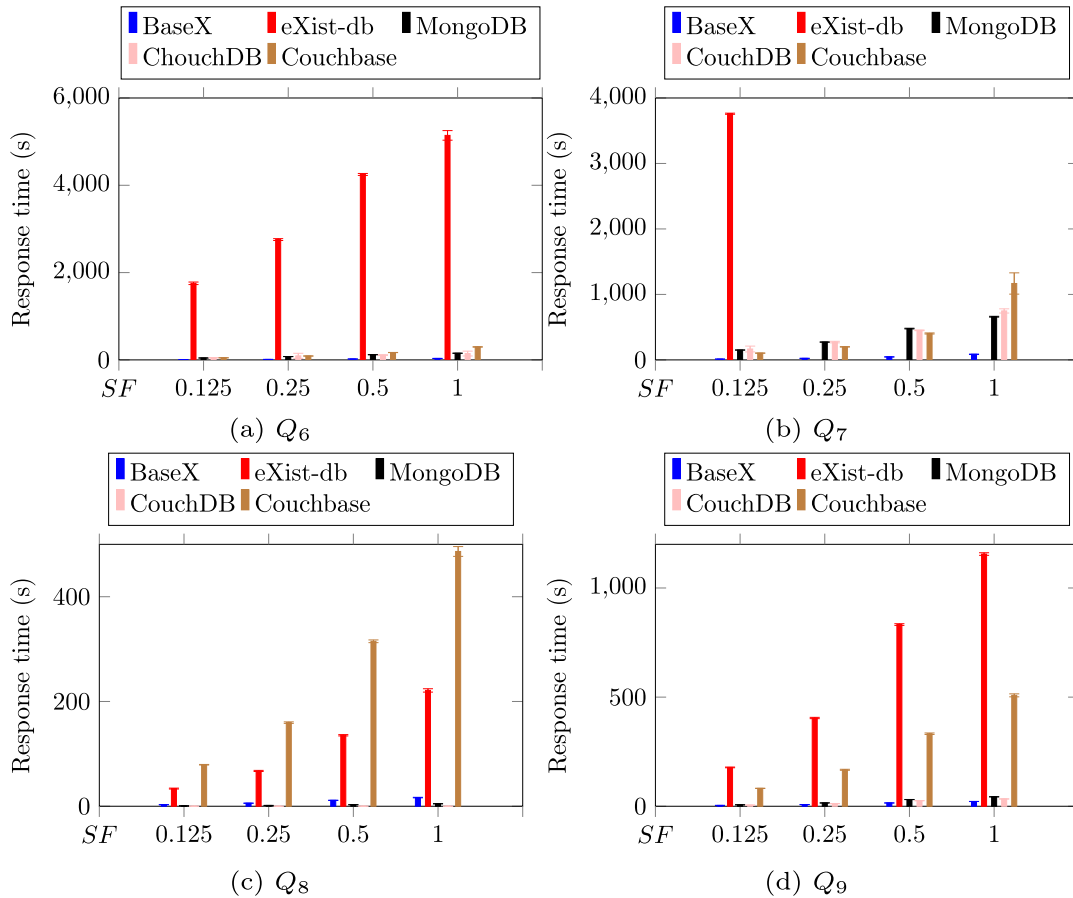


Fig. 6. Response time for aggregation queries. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

horizontal scaling. Thus, the aim of the paper is to examine single instance deployments.

There are many real-world scenarios where such single-instance deployment is preferred. As a first example, XDBMSes can be used for fast application development, analyzing and querying log data, or storing and retrieving IoT sensor data. XDBMSes are good candidates for storing large documents, managing long-running transactions, and querying hierarchical data structures in environments that require rapidly evolving schemes. Furthermore, these DBMSes are lightweight and do not require dedicated hardware, software, or a lot of resources. Thus, managing to lower resource costs at the data center site and enabling on-site data analysis and decision

making. Therefore, they can be utilized in Edge and Fog Computing with ease.

The creation of network islands due to faulty nodes is very common in the Edge/Fog environment. Even in the presence of well-defined recovery mechanisms, the formation of temporal network islands is unfavorable for sharding, as the overall latency increases if nodes go down and then up again. Hence, single-instance deployments are favored in these environments.

Another real-world scenario where such single-instance deployment can be used is for small to medium scale document management systems. These management systems are useful to smaller enterprises, where data is kept in the company due to GDPR (Eu-

ropean Union Legislation on General Data Protection Regulation). Moreover, as in many cases most of the data is in semi-structured formats, such as XML and JSON, single instance DODBMSes are a good candidate for storing and managing such documents. Thus, removing from the company's costs the maintenance of a data center.

It is also important to mention that the focus of our benchmark is on data retrieval and not on write operations because, in real-world applications, multiple techniques can be put in check to balance the write operations and minimize the workload. Moreover, data persistence can be achieved much later within a DBMS, depending on the workload and the systems write and logging mechanisms.

Furthermore, we loaded the data into the database using different methods. Because not all of the tested DODBMSes have their own data load tools, we developed our own data loading programs. By utilizing our data load programs and not native load DBMS functionalities, we added a new layer of complexity which decreases write performance. This makes the loading process to be dependent on external DBC (database connectors) implementations, and not on the DODBMS internal functionalities.

7. Conclusion

In this paper, we present an overview and comparison of DODBMSes that encode information using XML and JSON formats and propose a benchmark using filtering and aggregation queries on a heterogeneous dataset. For our experiments we chose three XDBMSes, i.e., BaseX, eXist-db, Sedna, and three JDBMSes, i.e., MongoDB, CouchDB, and Couchbase. These DODBMSes are open-source and free to use systems, whose license does not forbid benchmarking.

Our comparison focuses on key functionalities required by Big Data and IoT systems for storing and extracting information from large volumes of data. For this comparison, we also consider the transactions' properties of each DODBMSes, their in-memory capabilities, and how these systems deal with atomicity, consistency, isolation, durability with regards to operations such as accessing, modifying, and saving documents. We also present for each DODBMS its support for replication and partitioning of data and how it manages these Big Data requirements. Furthermore, we present the querying languages used for extracting information as well as the different types of indices provided by each DODBMS to improve retrieval response time.

The proposed benchmark uses different queries to emphasize the time performance of DODBMSes and highlights the capabilities of XDBMSes and JDBMSes. Furthermore, our solution proves its portability, scalability, and relevance by its design. The benchmark is portable, as it works on multiple systems. For this purpose, we compare the performance of several DODBMSes, i.e., BaseX, eXist-db, Sedna, MongoDB, CouchDB, and Couchbase. To demonstrate the scalability of our solution, we introduced *SF*, the scaling factor that generates an incremental growth in the data volume for our experiments. By increasing the queries' complexity together with the *SF* factor, we analyze the behavior of the systems from the scaling perspective. We observe that all the DODBMSes have a linear increase at runtime. Furthermore, BaseX proves to be a good choice when dealing with aggregations. Finally, our experimental results show that our benchmark is indeed relevant in comparing the runtime performance of different DODBMSes.

The performance tests provide some interesting and unexpected results. Among the XDBMSes, BaseX has the best overall performance. BaseX even outperforms the JDBMSes selected for this benchmark, i.e., MongoDB, CouchDB, and Couchbase, for three out of the four aggregation queries proposed. We observe that Couchbase has the overall worst performance among the JDBMSes. Sedna

outperforms CouchDB and Couchbase when dealing with filtering queries, but does not work for the aggregation queries. MongoDB has the overall best time performance for the filtering queries and it outperforms BaseX only for the aggregation query *Q₈*. eXist-db has some strange behavior when dealing with both filtering and aggregation queries. Also, it is highly dependent on the JVM, which needs to be tuned for each query, making this XDBMS hard to work with. However, we can assume that eXist-db works well on a query to query basis.

Following the results obtained by the benchmark, we can answer the three research questions and conclude that XDBMSes are still useful: their performance is as good as JDBMSes and they are good candidates for Big Data Management. Furthermore, XDBMSes are well-suited for several current real-world scenarios. Firstly, XDBMSes are reliable systems for storing large documents, managing long-running transactions, and querying hierarchical data structures in Edge/Fog environments (e.g., smart agriculture, healthcare wearables, etc.), as these types of DODBMSes are lightweight and do not require dedicated hardware, software, or a lot of resources. Secondly, XDBMSes can be used as small to medium scale document management systems in smaller enterprises, where data are kept in the company due to GDPR. Thirdly, in the case of Big Data analysis, they prove to be well-suited when the documents are in XML format, by removing the ETL (Extract, Transform, Load) processes from the storing, managing, and analysis pipeline.

As future work, we plan to add support for OLAP queries [41] on XML data and XML data in combination with other data [42,43] both in terms of performance and functionality. This will involve designing new sampling strategies and supporting more aggregation queries [42]. The sampling methods will include constraints on other labels and values contained in the records. Also, we aim to add more dimension for grouping [42], to boost the performance by lowering the query selectivity and performing query rewriting [43], and to add further grouping functionality [42].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The research presented in this paper was supported in part by the Danish Independent Research Council, through the *SEMIOTIC* project, and the *Robots and Society: Cognitive Systems for Personal Robots and Autonomous Vehicles (ROBIN)* project CCCDI-UEFISCDI grant No. PN-III-P1-1.2-PCCDI-2017-0734.

References

- [1] J. Han, H. E. G. Le, J. Du, Survey on NoSQL database, in: *International Conference on Pervasive Computing and Applications*, IEEE, 2011, pp. 363–366.
- [2] B.-K. Park, H. Han, I.-Y. Song, XML-OLAP: a multidimensional analysis framework for XML warehouses, in: *Data Warehousing and Knowledge Discovery*, Springer, 2005, pp. 32–42.
- [3] M. Stonebraker, U. Çetintemel, "One size fits all": an idea whose time has come and gone, in: *International Conference on Data Engineering*, IEEE, 2005, pp. 1–10.
- [4] C. Strauch, NoSQL databases, Tech. rep., Stuttgart Media University, 2011.
- [5] T. Zhu, D. Wang, H. Hu, W. Qian, X. Wang, A. Zhou, Interactive transaction processing for in-memory database system, in: *Database Systems for Advanced Applications*, Springer, Cham, 2018, pp. 228–246.
- [6] M.A. Qader, S. Cheng, V. Hristidis, A comparative study of secondary indexing techniques in lsm-based NoSQL databases, in: *International Conference on Management of Data, SIGMOD2018, ACM*, 2018, pp. 551–566.
- [7] A. Petrov, Algorithms behind modern storage systems, *Queue* 16 (2) (2018) 30:31–30:51, <https://doi.org/10.1145/3212477.3220266>.

- [8] D. Comer, Ubiquitous b-tree, *ACM Comput. Surv.* 11 (2) (1979) 121–137, <https://doi.org/10.1145/356770.356776>.
- [9] R. Cattell, Scalable SQL and NoSQL data stores, *SIGMOD Rec.* 39 (4) (2011) 12, <https://doi.org/10.1145/1978915.1978919>.
- [10] M. Stonebraker, SQL databases v. NoSQL databases, *Commun. ACM* 53 (4) (2010) 10, <https://doi.org/10.1145/1721654.1721659>.
- [11] R. Hecht, S. Jablonski, NoSQL evaluation: a use case oriented survey, in: *International Conference on Cloud and Service Computing*, IEEE, 2011, pp. 336–341.
- [12] F. Gessert, W. Wingerath, S. Friedrich, N. Ritter, NoSQL database systems: a survey and decision guidance, *Comput. Sci. Res. Dev.* 32 (3–4) (2016) 353–365, <https://doi.org/10.1007/s00450-016-0334-3>.
- [13] Z. Brahmia, H. Hamrouni, R. Bouaziz, XML data manipulation in conventional and temporal XML databases: a survey, *Comput. Sci. Rev.* 36 (2020) 100231, <https://doi.org/10.1016/j.cosrev.2020.100231>.
- [14] F. Bajaber, S. Sakr, O. Batarfi, A. Altalhi, A. Barnawi, Benchmarking big data systems: a survey, *Comput. Commun.* 149 (2020) 241–251, <https://doi.org/10.1016/j.comcom.2019.10.002>.
- [15] C.-O. Truică, E.-S. Apostol, J. Darmont, I. Assent, TextBenDS: a generic textual data benchmark for distributed systems, *Inf. Syst. Front.* (mar 2020), <https://doi.org/10.1007/s10796-020-09999-y>.
- [16] S.M. Freire, E. Sundvall, D. Karlsson, P. Lambrix, Performance of xml databases for epidemiological queries in archetype-based ehers, in: *Scandinavian Conference on Health Informatics*, Linköping University Electronic Press, 2012, pp. 51–57.
- [17] A. Schmidt, F. Waas, M. Kersten, M.J. Carey, I. Manolescu, R. Busse, XMark: a benchmark for xml data management, in: *International Conference on Very Large Databases VLDB*, Elsevier, 2002, pp. 974–985.
- [18] M. Nicola, I. Kogan, B. Schiefer, An XML transaction processing benchmark, in: *ACM SIGMOD International Conference on Management of Data*, ACM Press, 2007, pp. 937–948.
- [19] P. Atzeni, F. Bugiotti, L. Cabibbo, R. Torlone, Data modeling in the NoSQL world, *Comput. Stand. Interfaces* 67 (2020) 103149, <https://doi.org/10.1016/j.csi.2016.10.003>.
- [20] T. Fiebig, S. Helmer, C.-C. Kanne, G. Moerkotte, J. Neumann, R. Schiele, T. Westmann, Anatomy of a native XML base management system, *VLDB J.* 11 (4) (2002) 292–314, <https://doi.org/10.1007/s00778-002-0080-y>.
- [21] G. Pavlović-Lažetić, Native xml databases vs. relational databases in dealing with XML documents, *Kragujev. J. Math.* 30 (2007) 181–199.
- [22] E. Gallinucci, M. Golfarelli, S. Rizzi, Schema profiling of document-oriented databases, *Inf. Sci.* 75 (2018) 13–25, <https://doi.org/10.1016/j.is.2018.02.007>.
- [23] BaseX, BaseX documentation, http://docs.basex.org/wiki/Main_Page, 2020.
- [24] C. Grün, S. Gath, A. Holupirek, M.H. Scholl, XQuery full text implementation in BaseX, in: *Database and XML Technologies*, 2009, pp. 114–128.
- [25] W. Meier, eXist: An open source native xml database, in: *Web, Web-Services, and Database Systems*, Springer, 2003, pp. 169–183.
- [26] E. Siegel, A. Retter, eXist: A NoSQL Document Database and Application Platform, O'Reilly Media, Inc., 2014.
- [27] eXistdb, eXist-db documentation, <https://exist-db.org/exist/apps/doc/documentation>, 2020.
- [28] A. Fomichev, M. Grinev, S. Kuznetsov, Sedna: a native XML DBMS, in: *SOFSEM 2006: Theory and Practice of Computer Science*, Springer, 2006, pp. 272–281.
- [29] Sedna, Sedna documentation, <https://www.sedna.org/documentation.html>, 2020.
- [30] I. Taranov, I. Shcheklein, A. Kalinin, L. Novak, S. Kuznetsov, R. Pastukhov, A. Boldakov, D. Turdakov, K. Antipin, A. Fomichev, P. Pleshachkov, P. Velikhov, N. Zavaritski, M. Grinev, M. Grineva, D. Lizorkin, Sedna: native XML database management system (internals overview), in: *ACM SIGMOD International Conference on Management of Data*, SIGMOD'10, ACM, 2010, pp. 1037–1046.
- [31] K. Banker, P. Bakum, S. Verch, D. Garrett, T. Hawkins, MongoDB in Action, 2nd edition, Manning Publications Co., 2011.
- [32] MongoDB, Inc., MongoDB documentation, <https://docs.mongodb.com/>, 2020.
- [33] Apache CouchDB, CouchDB documentation, <https://docs.couchdb.org/en/stable/>, 2020.
- [34] J.C. Anderson, J. Lehnardt, N. Slater, CouchDB: Definitive Guide, O'Reilly Media, Inc., 2010.
- [35] B. Holt, Scaling CouchDB: Replication, Clustering, and Administration, O'Reilly Media, Inc., 2011.
- [36] G. Manyam, M.A. Payton, J.A. Roth, L.V. Abruzzo, K.R. Coombes, Relax with CouchDB — into the non-relational DBMS era of bioinformatics, *Genomics* 100 (1) (2012) 1–7, <https://doi.org/10.1016/j.ygeno.2012.05.006>.
- [37] M. Brown, Getting Started with Couchbase Server, Oreilly, 2012.
- [38] D. Vohra, Pro Couchbase Development, Apress, 2015.
- [39] M.A. Hubail, A. Alsuliman, M. Blow, M. Carey, D. Lychagin, I. Maxon, T. Westmann, Couchbase analytics, *VLDB Endow.* 12 (12) (2019) 2275–2286, <https://doi.org/10.14778/3352063.3352143>.
- [40] Apache Couchbase, Couchbase documentation, <https://docs.couchbase.com/home/index.html>, 2020.
- [41] D. Pedersen, K. Riis, T.B. Pedersen, A powerful and SQL-compatible data model and query language for OLAP, *Australasian Database Conference* 12 (2019).
- [42] T.B. Pedersen, D. Pedersen, J. Pedersen, Integrating XML data in the TARGIT OLAP system, *Int. J. Web Eng. Technol.* 4 (4) (2008) 495–533, <https://doi.org/10.1504/IJWET.2008.019945>.
- [43] T.B. Pedersen, X. Yin, Evaluating XML-extended OLAP queries based on physical algebra, *J. Database Manag.* 17 (2) (2006) 85–116, <https://doi.org/10.4018/jdm.2006040105>.